

本地决策模型做 Sub-agent 路由：Jeff 与 Laya 的对比、机制分析与 Stacking 级联

作者：Herr Wang 日期：2026-10-02 代码与数据：github.com/tiw/jeff-laya-routing-study 环境：Apple M4 Pro / 48GB / macOS 26.6.2, MLX 后端 (Jeff)、laya-mlx 0.2.0 (Laya)、Ollama qwen3.8:27b-mxpf8 (大模型基线)

摘要

把 LLM 请求路由到合适的执行者 (sub-agent / 工具 / 大模型) 是多 agent 系统的核心开销之一。本文在「角色 Sub-agent 路由」任务上，对两个本地开源决策模型——解码器路线的 Jeff-Qwen3.5-2B 与编码器路线的 Laya (ModernBERT-large, 421M)——做了系统的对比评测、控制变量机制实验、stacking 元模型训练与跨分布 (OOD) 泛化测试，并交付了一个可运行的级联路由器。主要发现：(1) Jeff-2B 以 92ms、近零成本达到 85% 路由准确率，其置信度是可靠的升级信号，简单置信度级联即可达 97.5%；(2) 两模型错误高度互补，机制可解释为「解码器动作驱动 + 代码话题先验」与「编码器实体锚定」的差异，并被语序翻转、话题梯度、遮罩消融三组实验证实；(3) 将两模型的完整概率向量拼接后训练 softmax 回归元模型 (stacking)，分布内 5 折 CV 达 95.7% (零升级成本)，OOD 测试 94.0%，配合置信度级联后 OOD 99.2%、升级率仅 23.4%，在准确率和成本上同时优于手写阈值规则；(4) 元模型学到的非对称信任权重与错误分析互相印证——互补性必须条件化才能兑现，线性概率平均无增益。所有数据 (540 条带金标准) 与代码开源。

关键词：LLM 路由、决策模型、stacking 集成、级联推理、本地部署、sub-agent

1. 引言

多 agent 系统中，「这个请求该给谁处理」是一个高频、低延迟要求的判定：它挡在所有执行之前，错了后面全错，但它本身不值得花一次大模型调用的成本和秒级延迟。工程上常见的方案是手写规则或让大模型兼任裁判，前者脆、后者贵。

2026-09-28 登上 Hacker News 首页的 Jeff (firelex/jeff) 提供了一条新路径：它是 TypeSafe System One 决策模型 Jev 的本地开源兼容实现——用 Qwen3.5 / Gemma 4 微调出的零样本决策模型，单次前向传播直接输出每个候选选项的校准概率，不生成文本、不需要解析，请求格式与 Jev 的 /v1/systemone 完全兼容。另一条路线是 convaiinnovations/laya (laya-mlx)：基于 ModernBERT-large 编码器的类型化决策模型，20ms 级延迟、842MB 权重。

本文回答四个问题：

- RQ1 (能力水位)：在本地的角色路由任务上，两个小模型各能到多少？置信度能否作为「升级给大模型」的信号？
- RQ2 (机制差异)：编码器与解码器两条路线的错误模式有何系统差异？能否用架构特性解释？
- RQ3 (组合收益)：两者的互补性能否通过 stacking (训练元模型学「什么情况下信谁」) 兑现？相对手写阈值和线性平均有多少增益？
- RQ4 (泛化性)：以上结论在分布外真实请求上是否成立？

2. 背景与相关概念

决策模型 vs 对话模型。决策模型把「情境 + 候选选项」映射为选项上的校准概率分布，输出端是分类头/选项字母上的 softmax，而非自由文本。它适合路由、审核、分流这类「判断」型负载，延迟和成本比对话模型低一到两个数量级。

Stacking (Wolpert, 1992)。集成学习三大范式之一：基模型的预测不作为最终答案，而是作为新特征喂给元模型 (meta learner)，由元模型输出最终判决。与投票 (假设每个模型永远同样可信) 不同，stacking 学

的是「什么情况下该信谁」的条件化信任权重。MoA (Mixture-of-Agents) 是其在 LLM agent 上的对应物；本文的路由层即元学习器的角色，手写阈值规则是其零训练版本。

两个被测模型：

	Jeff-Qwen3.5-2B (v1.2)	Laya typed-decisions (421M)
架构	Qwen3.5 解码器 SFT，单向注意力，选项字母上压概率	ModernBERT-large 编码器 + 分类头，双向注意力
权重	4.4GB (Apache 2.0)	842MB
请求格式	Jev /v1/systemone (choice/noul/score)	laya-mlx CLI / Python API (choice/noul/score)
官方定位	80% 高频判断本地做，20% 升级大模型	极端吞吐场景的 20ms 决策

3. 实验设置

任务。四分类角色路由：researcher / coder / writer / reviewer。输入是用户自然语言请求 (state)，两家小模型用完全相同的 choice 问题与 criteria 文案；27B (qwen3.8:27b-mx8, temperature 0) 用等价 chat prompt 做 zero-shot，扮演独立裁判生成金标准。

金标准规则：多意图请求取第一步动作；「解释为什么 X 出问题」型归入 researcher (此类本身有争议，单独标注 disputed)。

数据集 (全部随仓库开源，540 条)：

集合	规模	用途	构造方式
主数据集	40	三方对比	20 英文 + 14 中文 + 6 边界案例，人工撰写
E2 正式版	240	话题梯度 + stacking 训练	4 档技术度 × 20 话题 × 3 动作框架 (explain/fix/write)，受控模板
set_a_ood	100	OOD 测试	真实分布请求，四类各 25，无框架模板
set_b_reviewer	50	OOD 测试	reviewer 专项 (40 正例 + 10 hard negative)
set_c_diagnosis	40	OOD 测试	「解释为什么」诊断边界集 (含 8 条 disputed)
set_d_multi_intent	40	OOD 测试	多意图长请求
set_e_vague	30	OOD 测试	模糊/欠指定 (12 条零上下文 gold 为占位，不计准确率)

协议：stacking 元模型仅用 280 条分布内数据 (40 主数据集 + 240 E2) 训练，5 折分层交叉验证；OOD 测试在 5 个新专题集上进行 (n=248 valid)，两者严格隔离，不存在测试折泄漏。

级联的记账口径：升级后按 oracle 计 (假设 27B 全对；40 例实测为真但样本小，第 10 节讨论)。

4. RQ1：三方对比（40 例主数据集）

指标	Laya-421M	Jeff-2B	Qwen3.8-27B
总准确率	70.0%	85.0%	100%
英文 / 中文 / 边界	70 / 71 / 67%	75 / 93 / 100%	100%
中位延迟	20ms	92ms	913ms
置信度（判对 / 判错均值）	0.143 / 0.077（不可用）	0.797 / 0.592（可用）	—

按金标准类别：coder 两家都 11/11；reviewer Jeff 8/9 vs Laya 3/9（Laya 最大短板）；researcher 7/10 vs 6/10；writer 8/10 vs 8/10。

置信度作为升级信号（级联模拟）：

- Jeff 置信 $< 0.7 \rightarrow$ 升级 27B：升级率 35%，级联准确率 97.5%
- Jeff+Laya 集成（一致 $\rightarrow$ 自动，不一致 $\rightarrow$ 升级）：升级率 35%，级联 95.0%；一致时准确率 92.3%
- Laya 单独按置信度升级：信号弱（判对/判错置信度几乎不可分），不可用

即：65% 的请求以 92ms、近零成本解决，整体 97.5%。功能冒烟测试（5 用例：子代理路由、注入守卫、缺陷严重度、金融情绪、中文客服）方向全部正确，确认 2B 的真实水位——英文任务自信（0.99），中文与注入检测置信中等（0.6-0.75）。

5. RQ2：错误互补性与机制分析

5.1 错误互补性

16 道分歧题：Jeff 独对 10、Laya 独对 4、双错 2（双错均为「解释为什么 X 出问题」型，金标准本身有争议，视为标注噪音）。

- Laya 被实体名词锚定：错过“Check this SQL query…” / “Audit this Docker setup” / “简历检查” / “技术方案评估”，全部按制品归类（代码制品 $\rightarrow$ coder，文字制品 $\rightarrow$ writer），丢失了「要对这东西做什么」这一层。
- Jeff 被代码话题先验带走：错过“Review my pull request”、“MQTT 和 Kafka 区别”、“changelog 起草”、“训练 loss 为什么抖”——凡话题沾技术边即判 coder，哪怕动作是 explain/compare/draft。
- Laya 反超 Jeff 的恰好是动词外露的模板句（“Review my…”、“Draft the…”），接近其 typed-decisions 微调工作流的训练分布。

5.2 架构层解释

Laya 是编码器：双向注意力、MLM 预训练，一次性看到全句，句中最具体的实体主导表征——「名词开头、动作在后」的语序下动作词被埋，归类退化为按话题。Jeff 是解码器：单向注意力、下一词预测、SFT 教它找动作动词，所以 reviewer 类远好于 Laya；但 Qwen 底座的代码先验太强，单 epoch SFT 只调整不替换。

5.3 三组控制变量实验

E1 语序翻转（主动 $\leftrightarrow$ 被动，10 组）——原假设被推翻：两家翻转率均 1/10，对语序都鲁棒。仅有的翻转反而支持机制假说：Jeff 翻在技术名词题（PostgreSQL/MySQL），Laya 翻在动作名词化的被动句（“needs an audit”）。

E2 话题梯度扫描——强证实。固定动作「解释一下为什么 X」，X 从技术度 0（吉他生锈）到 3（PG 慢查询）：Jeff P(coder) 0.007→0.191→0.567→0.434；换成「写一篇介绍」框架后全部压回 0.01-0.09。代码先验 = 话题技术度 × 动作类型的交互项。

E3 遮罩消融（8 题 × 原句/遮实体/遮动词）——证实。Jeff 遮动词掉 2 分（5/8→3/8），遮实体无感 → 靠动作动词；Laya 遮实体反而 +3 分（1/8→4/8）→ 实体正是误导锚点，遮掉即解除；遮动词无感 → 本来就没用动词。

5.4 E2 正式版：240 条梯度数据

4 档技术度 × 20 话题 × 3 框架 (explain/fix/write)，共 240 条。完整梯度图见 assets/exp2_gradient.png，关键格子：

框架（金标准）	档位	Jeff P(coder) (acc)	Laya P(coder) (acc)
explain (researcher)	非技术	0.01 (100%)	0.26 (5%)
	web/开发	0.57 (35%)	0.39 (5%)
fix (coder)	非技术	0.04 (0%)	0.44 (95%)
	web/开发	0.85 (100%)	0.54 (100%)
write (writer)	全部	0.02-0.06 (100%)	0.25-0.31 (85-100%)

三个新发现：

- 1. Jeff 的「代码门」是双面的：explain× 技术 过度触发 (acc 仅 35%)，fix× 非技术 反向漏触发 (非技术档 acc 0%，28 条全派 researcher) ——它的 coder 概念是「修代码的人」而非「修东西的人」，需要动作与话题同时成立。
- 2. Laya 的平是全面的平：所有条件下 P(coder) 都在 0.25-0.54，std 仅 ±0.07；Jeff 同条件 std 高达 ±0.26。即「平时靠谱、个别题剧烈摇摆」vs「永远温吞均匀分布」。
- 3. Laya 在 explain 框架的错误是语义盲区：80 次判定 coder 46 / writer 21 / researcher 仅 9——「解释一下为什么」几乎从不映射到 researcher。

6. RQ3: Stacking 元模型

方法：特征 = [Jeff 4 维概率 || Laya 4 维概率] (8 维)，标签 = 金标准类别；softmax 回归 numpy 手写（无 sklearn），5 折分层交叉验证。级联版本取训练集上使升级率 ≈22.5%（与手写规则对齐）的置信度阈值，升级后按 oracle 计。

分布内结果（280 条，5-fold CV）：

方案	准确率	升级率
Jeff 单独	78.6%	—
Laya 单独	66.4%	—
概率平均（线性意见池）	78.2%	—
手写级联（Jeff conf<0.7 → 27B）	86.8%	22.5%
元模型单独	95.7%	0%
元模型级联 · 同预算	99.6%	22.9%

三个要点：

1. 学习版裁判在零升级成本下已超过手写级联：95.7% > 86.8%（后者还花了 22.5% 的 oracle 升级）。同预算对比再 +12.8pt。
2. 学到的非对称信任权重与第 5 节错误分析互相印证：writer/reviewer 类大权重落在 Jeff 概率上（Jeff reviewer 8/9、writer 8/10），researcher 类参考 Laya 信号；线性概率平均（78.2%）完全不增长——**互补性必须非线性/条件化才能兑现**，这正是 stacking 相对投票的本质优势。
3. 诚实警告：240/280 条来自受控框架，测试折与训练折同分布，95.7%/99.6% 是分布内乐观估计。RQ4 专门验证跨分布泛化。

7. RQ4: OOD 泛化测试

协议：仅用 280 条分布内数据训练 → 260 条新专题集测试（E 集 12 条零上下文 gold 为占位，准确率计 248 条 valid）。

方案（OOD 测试，n=248）	准确率	升级率
Jeff 单独	87.9%	—
Laya 单独	64.1%	—
概率平均组合	88.7%	—
手写级联（conf<0.7）	98.4%	33.5%
元模型（分布内训练 → OOD 测试）	94.0%	0%
元模型级联（阈值取自训练集）	99.2%	23.4%

逐集（元模型 OOD 测试）：A 真实分布 95.0%、B reviewer 100%、C 诊断 97.5%（Jeff 单独仅 70%——E2 学到的「explain 框架下 Jeff 的 coder 信号打折」成功迁移到了真实诊断句）、D 多意图 95.0%、E 可推断子集 61.1%（极短间接语义仍是共同短板，但级联会兜底）。

结论：

1. **学习版裁判跨分布泛化成立**：分布内学到的非对称信任权重在真实分布上 +6.1pt（94.0% vs 87.9%），零额外推理成本。第 6 节的「分布内乐观估计」担心被部分解除。
2. **元模型级联全面占优**：准确率 99.2% vs 98.4%，升级率 23.4% vs 33.5%——准确率和成本双赢。且 Jeff 在 OOD 上置信度天然下移（升级率从 22.5% 升至 33.5%），符合「请求越非标越保守」的期望行为。

8. 工程交付：级联路由器

code/cascade_router.py (stdlib HTTP, ~110 行, 端口 8767): 接收 POST /route {state} → 并行查 Jeff (:8766) 与 Laya 本地权重 → 概率拼接过元模型 → meta conf < τ=0.7629 则升级本地 27B (Ollama)。

端到端测试（4 类请求行为全部正确）：

请求	判决	升级	延迟
「帮我把爬虫改成增量抓取」	coder (conf 0.7322)	是 → 27B 补判 ✓	40.9s (27B 首载) / 1.8s (重跑)
「解释为什么训练 loss spike」	researcher 0.9603	否 ✓	19.8s (27B 常驻时内存压力)
「Audit this Terraform module」	reviewer 0.9587	否 ✓	1.8s
「帮我看看这个。」(零上下文)	低置信	是 → 27B 判 reviewer ✓	21.2s

两个已知问题（诚实披露）：

1. $\tau=0.7629$ 偏激进：清晰的 coder 请求若 Laya 概率摊得开（如 writer 0.33），meta 置信会被拖到阈值下造成误升级。调低 τ 或对 Laya 特征降权是后续优化方向。
2. 27B 常驻的内存代价：统一内存压力下小模型延迟从 92ms 基线升至 ~1.8s。生产部署需权衡大模型常驻 vs 按需冷启动。

9. 讨论：工程结论

1. 当前最优单机方案：Jeff 做主路由 + stacking 元模型判决 + 置信度级联。~76% 请求毫秒级零成本解决，整体 99.2% (OOD)，升级率 23.4%。
2. Laya 的定位：20ms/842MB 适合极端吞吐场景；置信度不可用于升级信号（只能信排序）；reviewer 类需领域微调。
3. 微调方向（若做）：Jeff 需「技术话题 × 非代码动作」反例对；Laya 需 reviewer 语料 + 名词开头语序变体。两家的训练数据均全合成、可不出内网。
4. 标注纪律：「解释为什么」类请求的金标准本身模糊，评测时需固定规则并披露 disputed 标注。

10. 局限

- 40 例主数据集偏小，金标准为单人标注（27B 独立应用同规则 100% 通过，部分缓解）。
- E2 话题由研究者撰写、档位划分粗、单模型版本；每格 $n=20$ ，属方向性证据。
- 级联记账假设升级 oracle（27B 全对），40 例实测为真但样本小；27B 自身在更难分布上的错误率未测。
- OOD 各专题集仍有构造痕迹（人工撰写），与真实生产流量存在差距；极短间接语义（set_e 可推断子集 61.1%）仍是共同短板。
- 所有延迟为单机同进程 HTTP 测量，未测并发。
- τ 取自训练集升级率对齐，未在真实流量上校准。

11. 结论

本文证明了本地决策模型 + stacking 元学习器 + 置信度级联的三层路由架构在角色路由任务上成立：毫秒级延迟承担 ~76% 流量，99% 级准确率，升级成本可控。机制实验显示编码器/解码器的互补性不是玄学而是可测量、可解释、可被元模型条件化利用的结构差异。对多 agent 系统设计的启示：路由层值得被当作一个独立的、可学习的组件对待，而不是手写规则或大模型兼任的附属品。

参考文献

1. firelex/jeff —Jev 兼容的本地开源决策模型. <https://github.com/firelex/jeff>（2026-09-28 登 Hacker News 首页）
2. TypeSafe Jev —System One 决策模型（云端 API）. <https://typesafe.com/jev>
3. convaiinnovations/laya —typed-decisions 编码器决策模型. <https://huggingface.co/convaiinnovations/laya>
4. Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241-259.
5. Qwen3.5 技术报告（Jeff 底座模型）.